

Relatório 4 - Programação Paralela

1. Enunciado

Implemente uma versão sequencial de um programa para o cálculo de integrais definidas utilizando o método do trapézio. O programa deve permitir calcular a integral de uma função $f(x)$ em um intervalo $[a,b]$, utilizando n subdivisões.

Em seguida, analise o código e discuta quais partes podem ser paralelizadas, identificando também os possíveis desafios ou problemas relacionados à paralelização desse problema. Considere, por exemplo, questões como dependências entre operações, acesso concorrente a variáveis compartilhadas e a necessidade de sincronização ou redução dos resultados parciais.

2. Objetivo da Tarefa

O objetivo desta atividade é aplicar os conceitos de programação paralela no cálculo numérico da integral definida de uma função $f(x)$ usando o **Método do Trapézio**. A tarefa visa identificar as regiões do algoritmo suscetíveis ao paralelismo de dados (laço de repetição), analisar os desafios de sincronização na acumulação dos resultados parciais e demonstrar a solução via **OpenMP** utilizando a cláusula **reduction**.

3. Metodologia

A solução foi desenvolvida na linguagem C utilizando a biblioteca OpenMP (`<omp.h>`). O problema foi modelado para integrar numericamente a função $f(x) = x^2$ no intervalo de $a = 0.0$ a $b = 10.0$ com $n = 1.000.000$ de subdivisões ($h = \frac{b-a}{n}$).

A estratégia de paralelização baseou-se em:

1. **Cálculo Sequencial dos Extremos:** A avaliação das extremidades $f(a)$ e $f(b)$ é feita uma única vez antes da região paralela.
2. **Distribuição do Laço de Repetição:** O laço responsável por avaliar os pontos intermediários $x_i = a + i \cdot h$ foi paralelizado utilizando a diretiva `#pragma omp parallel for`.
3. **Mecanismo de Redução:** Para evitar a condição de corrida na variável acumuladora `soma`, utilizou-se a cláusula `reduction(+:soma)`.

A compilação no macOS seguiu o padrão para habilitar a OpenMP via Clang: `clang -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include -L$(brew --prefix libomp)/lib -lomp tarefa4.c -o tarefa4`

4. Código-Fonte

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

// compilar: clang -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include -L$(brew --prefix libomp)/lib -lomp tarefa4.c -o tarefa4

// Função f(x) = x^2
double f(double x) {
    return x * x;
}

int main() {
    double a = 0.0;
    double b = 10.0;
    int n = 1000000;
```

```

double h = (b - a) / n;
double soma, integral;

// Extremos da fórmula (fora do laço, calculado apenas uma vez)
soma = (f(a) + f(b)) / 2.0;

// =====
// PARALELIZAÇÃO DO LAÇO PRINCIPAL
// Usamos reduction(+:soma) para que cada thread
// some seus trapézios parciais com segurança.
// =====
#pragma omp parallel for reduction(+:soma)
for (int i = 1; i < n; i++) {
    double x_i = a + i * h;
    soma += f(x_i);
}

integral = h * soma;

printf("O valor da integral de %f a %f com %d subdivisões eh: %f\n", a, b, n, integral);

return 0;
}

```

5. Resultados e Análises

5.1 Saída Observada no Terminal

Ao executar o código paralelizado para $n = 1.000.000$ subdivisões, obtém-se o resultado exato esperado pela análise matemática:

O valor da integral de 0.000000 a 10.000000 com 1000000 subdivisões eh: 333.333333

(Nota: A integral analítica de $\int_0^{10} x^2 dx = \left[\frac{x^3}{3} \right]_0^{10} = \frac{1000}{3} \approx 333,333333$, o que confirma a corretude do cálculo paralelo).

5.2. Partes Paralelizáveis vs. Sequenciais

- **Parte Sequencial (Não paralelizada):** A definição do intervalo, o cálculo da largura h , o cálculo inicial dos dois pontos extremos $(f(a) + f(b))/2.0$ e a multiplicação final por h . Essa fração de código possui custo computacional $O(1)$, não compensando a sobrecarga (*overhead*) de criação de threads.
- **Parte Paralelizável (Paralelismo de Dados):** O laço for que itera 1.000.000 de vezes. Cada iteração realiza a avaliação independente do ponto $x_i = a + i \cdot h$ e calcula $f(x_i)$. Como o cálculo de x_i depende apenas do índice i , do ponto inicial a e de h (constantes), **não existe dependência de dados entre as iterações** (não há dependência *Loop-Carried*).

5.3. Desafios da Paralelização e Solução Técnica

1. Condição de Corrida (Race Condition):

- **O Problema:** A operação `soma += f(x_i)` atualiza uma variável global/compartilhada. Se múltiplas threads tentarem ler e escrever em `soma` simultaneamente sem sincronização, ocorrerá perda de atualizações e o valor final da integral estará totalmente incorreto.
- **Tentativa Ineficiente (#pragma omp critical ou atomic):** Proteger a linha `soma += f(x_i)` com uma seção crítica forçaria as threads a executarem a adição de forma serializada. Isso gera uma sobrecarga massiva (*lock contention*) e destrói o ganho de desempenho.

2. A Solução com Redução (reduction(+:soma)):

- A cláusula `reduction(+:soma)` resolve o problema de maneira eficiente em termos de hardware:
 - O OpenMP cria automaticamente uma **cópia privada** da variável `soma` para cada thread (inicializada com 0).
 - Cada thread calcula e acumula o resultado dos seus trapézios na sua variável privada, em velocidade máxima e sem nenhuma colisão de memória.
 - Ao final do laço paralelo, a biblioteca realiza uma combinação atômica/em árvore das somas parciais de todas as threads, somando-as de volta na variável `soma` principal.

6. Conclusão

A atividade demonstra a eficácia do paralelismo de dados aplicado a métodos numéricos. O cálculo da integral pelo método do trapézio é um problema altamente idêntico e independente (*embarrassingly parallel*), onde a única barreira crítica é a acumulação do resultado final. A utilização da diretiva `#pragma omp parallel for reduction(+:soma)` permitiu distribuir as iterações entre múltiplos núcleos de processamento de forma limpa e segura, eliminando a condição de corrida sem introduzir o gargalo de sincronizações atômicas manuais na região crítica.